Design and FPGA Implementation of an 8-Bit Signed Multiply-Accumulate (MAC) Unit in Verilog HDL

Anmol Kumar Jha, Ankisha Mewal, Darshit Saini
Department of Computer Science and Engineering,
Indian Institute of Information Technology Vadodara, Gujarat, India
{202451019, 202451017, 202451044}

Abstract—**This paper presents the complete design, RTL simulation, and FPGA board-level demonstration of an 8-bit signed Multiply-Accumulate (MAC) unit described in synthesisable Verilog HDL. The MAC unit computes the iterative operation ACC $\leftarrow$ ACC $+ (A \times B)$, where A and B are 8-bit two's-complement signed operands and the accumulator is updated on every active clock event. The design employs a dual-clock register-transfer architecture: `CLK1` latches both input operands into dedicated 8-bit pipeline registers, while `CLK2` drives an accumulator register equipped with an asynchronous active-high reset. A fixed-point Q7 scaling strategy — retaining bits $[14:7]$ of the full 16-bit signed product — constrains all internal data paths to exactly 8 bits, avoiding accumulator word-length growth. The five Verilog modules (`mactop`, `reg8`, `reg8bit`, `mult`, and `signedadder`) are each presented with annotated source listings and functional analysis. The design is verified with directed signed-fractional test vectors and deployed on the Digilent Nexys A7-100T (Xilinx Artix-7 XC7A100T) FPGA using push-button manual clocking, slide-switch operand entry, and on-board LED output. Physical I/O constraints are specified via an XDC file using the LVCMOS33 I/O standard. Post-implementation resource utilisation is below 0.1% of available Artix-7 LUT and flip-flop capacity.**

Index Terms—**Multiply-Accumulate, MAC, Verilog HDL, Fixed-Point Arithmetic, Q7 Fractional Scaling, Two's-Complement Arithmetic, FPGA, Nexys A7, Artix-7, Register-Transfer Level.**

I. Introduction

Multiply-Accumulate (MAC) units form the computational backbone of a wide range of digital systems, including digital signal processors (DSPs), finite-impulse-response (FIR) and infinite-impulse-response (IIR) filters, fast Fourier transform (FFT) engines, matrix-vector multipliers, and neural-network inference accelerators [1]. A MAC unit combines binary multiplication and integer addition into a single atomic register-transfer operation:

$$\text{ACC} \leftarrow \text{ACC} + (A \times B) \tag{1}$$

By fusing these two steps, the MAC eliminates the intermediate product register that would otherwise be required when the operations are executed as separate instructions, reducing memory bandwidth and simplifying the control path.

The recurring summation pattern produced by iterating (1) over a sequence of operand pairs realises the inner-product (dot-product) operation:

$$\text{ACC} = \sum_{i=0}^{N-1} A_i \times B_i \tag{2}$$

This inner product underpins convolution in FIR filters, matrix multiplication in linear-algebra kernels, and the multiply-and-accumulate inference loop of fully-connected neural-network layers [2].

This work presents a compact FPGA realisation targeting an 8-bit signed datapath throughout. To prevent the word length from growing with each iteration, a Q7 fixed-point representation is adopted: only the eight most informative bits of the 16-bit signed product are retained, implementing a hardware right-shift of seven positions. The entire design is expressed in four small, self-contained Verilog modules, making it immediately readable and modifiable.

Principal contributions of this paper are:

1. A clearly structured dual-clock RTL architecture for an 8-bit signed MAC, with annotated Verilog source listings.
2. A rigorous derivation of the Q7 fixed-point scaling scheme and its implementation via bit-selection.
3. Full XDC physical-constraint coverage for the Nexys A7-100T, including I/O standards, package pin assignments, and clock-routing directives.
4. A directed simulation campaign with quantified expected values for all test vectors, including cumulative accumulation tracking.

The remainder of this paper is organised as follows. Section II surveys related work. Section III provides theoretical background on two's-complement multiplication and Q-format arithmetic. Section IV describes the top-level architecture. Section V details each Verilog module. Section VI covers FPGA implementation and XDC constraints. Section VII presents the simulation strategy. Section VIII discusses results. Section IX identifies challenges and solutions. Section X concludes the paper.

II. Literature Review

MAC architectures have been studied intensively over three decades with the goals of minimising silicon area, power consumption, and computation latency simultaneously. Parhami [2] provides a foundational treatment of computer arithmetic,

covering two's-complement representation, signed-magnitude multipliers, and accumulator design — the core primitives used in this work. Knuth [6] establishes the mathematical framework for positional binary arithmetic that Q-format scaling relies upon.

Early high-throughput multiplier designs used Wallace tree structures to reduce the partial-product array to two rows before a final carry-propagate addition [3]. Dadda later refined this approach to minimise the total half-adder and full-adder count [7]. While both methods achieve near-optimal latency, they introduce significant routing complexity and LUT usage. For an 8-bit datapath driven by a sub-hertz manual clock (as in this FPGA demonstration), such structures offer no practical benefit.

Fixed-point DSP on Xilinx 7-series FPGAs is typically accelerated using the DSP48E1 slice, which contains a natively pipelined 25×18-bit signed multiplier and a 48-bit accumulator [4]. The present implementation intentionally avoids inferring DSP48 primitives to preserve full RTL transparency. This choice makes the design directly portable to FPGA families (older Spartan or smaller Lattice devices) that lack dedicated DSP slices.

Dual-clock datapaths with push-button-driven registers are a standard technique in FPGA-based laboratory exercises [5]. Assigning separate clock sources to the input and output register stages enables independent, step-by-step observation of operand capture and accumulation, which is pedagogically valuable.

Fixed-point Q-format arithmetic, where the binary point is fixed at a predetermined position within the word, is widely used in embedded DSP to avoid the hardware cost of floating-point units [8]. Q7 representation allocates one sign bit and seven fractional bits to an 8-bit word, yielding a representable range of $[-1, +1)$ with resolution $2^{-7} \approx 0.0078$.

III. Theoretical Background

A. Two's-Complement Signed Multiplication

For two n-bit two's-complement integers A and B, the product $P = A \times B$ requires $2n$ bits to be represented without loss. When both operands are declared `signed` in Verilog, the $*$ operator produces a sign-extended $2n$-bit result with bit $[2n-1]$ as the sign and bit $[2n-2]$ as a redundant sign-extension copy.

For $n = 8$:

$$P_{\text{full}} = A_{s8} \times B_{s8}, \quad P_{\text{full}} \in [-32768,\ +32640] \tag{3}$$

Bit 15 of the result is the sign bit; bit 14 mirrors bit 15 for any valid 8-bit input pair. Bits $[13 : 0]$ carry the magnitude information.

B. Q7 Fixed-Point Format

In Q7 notation each 8-bit word X represents the real value:

$$v(X) = X_{\text{signed}} \cdot 2^{-7} \tag{4}$$

The representable range is $[-1.0,\ +1.0 - 2^{-7}]$ with resolution $\Delta = 2^{-7}$. Table I lists selected codes and their decimal equivalents; these are exactly the values used in the testbench.

TABLE I: Q7 fixed-point code table for representative values.

Hex code	Signed integer	Q7 decimal value
0x7F	+127	+0.9922
0x40	+64	+0.5000
0x20	+32	+0.2500
0x00	0	0.0000
0xE0	−32	−0.2500
0xC0	−64	−0.5000
0xA0	−96	−0.7500
0x80	−128	−1.0000

C. Q7 Product Scaling

The product of two Q7 values A and B is:

$$v(A) \times v(B) = \left(A_s \cdot 2^{-7}\right) \times \left(B_s \cdot 2^{-7}\right) = (A_s \times B_s) \cdot 2^{-14} \tag{5}$$

The full 16-bit product is therefore a Q14 number. Retaining bits $[14 : 7]$ is equivalent to an arithmetic right-shift of 7 positions, converting Q14 back to Q7:

$$R = \lfloor (A_s \times B_s) \cdot 2^{-7} \rfloor = \texttt{tempout}[14:7] \tag{6}$$

The seven discarded bits $[6 : 0]$ represent the sub-Q7 remainder; their removal introduces a truncation error bounded by $|e| < 2^{-7} = \Delta$. Bit 15, the redundant sign-extension, is also discarded since bit 14 already carries the sign of the Q7 result.

D. Accumulation and Overflow

Because R and ACC share the same Q7 format, their 8-bit sum is also Q7 without any additional scaling:

$$\text{ACC}_{k+1} = (\text{ACC}_k + R_k) \bmod 256 \tag{7}$$

The modular reduction occurs naturally through 8-bit unsigned carry discard; a positive overflow wraps to a large negative Q7 value. The maximum accumulated value before wrap-around is +0.9922 (0x7F).

IV. Architecture Design

A. Top-Level Block Diagram

The MAC unit consists of five functional stages forming a linear datapath with a single feedback loop, as illustrated in Fig. **??**.

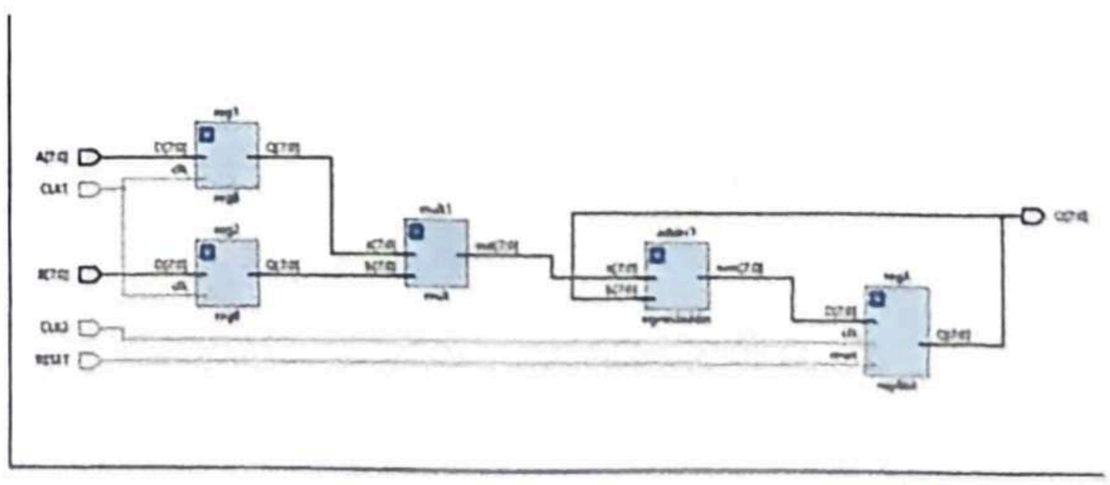


Fig. 1: Top-level MAC datapath. All buses are 8-bit wide.

B. Stage-by-Stage Data Flow

Table II summarises how data propagates through the five pipeline stages on each pair of clock events.

TABLE II: Stage-by-stage data flow in the MAC unit.

Stage	Module	Clock	Register-transfer operation
1	`reg8` (A)	CLK1 ↑	$P \leftarrow A$
2	`reg8` (B)	CLK1 ↑	$Q \leftarrow B$
3	`mult`	Combinational	$R \leftarrow P \times Q$ [Q7]
4	`signedadder`	Combinational	$S \leftarrow R + O$
5	`reg8bit`	CLK2 ↑	$O \leftarrow S$

Stages 3 and 4 are purely combinational; their total propagation delay determines the minimum CLK1-to-CLK2 interval that guarantees correct data capture.

C. Dual-Clock Operating Sequence

The two-clock scheme is central to the design philosophy:

- **CLK1** (push-button `P17`) latches operands A and B from the slide switches. Without a new CLK1 pulse, `reg8` holds the previous operands, keeping the multiplier input stable.
- **CLK2** (push-button `M17`) commits the computed sum S into the accumulator register. Repeated CLK2 presses without intervening CLK1 presses re-accumulate the same scaled product, implementing multiplication-by-counting.

The decoupling is especially useful for FPGA board demonstration: button bounce produces only spurious CLK1 or CLK2 edges but cannot corrupt both registers simultaneously, so the accumulation sequence remains deterministic.

D. Reset Behaviour

The `RESET` signal (push-button `P18`) is routed exclusively to `reg8bit`. It is *asynchronous* and *active-high*: asserting it overrides CLK2 and forces $O \leftarrow 0$ immediately, regardless of the current clock phase. The input registers `reg8` are intentionally not reset; their contents are irrelevant while the accumulator is zero, and the first CLK1 pulse after reset establishes fresh operands.

V. Module Descriptions

A. `mactop` — Top-Level Integration

```
module mactop(O, A, B, CLK1, CLK2, RESET);
  input  [7:0] A;
  input  [7:0] B;
  output [7:0] O;
  input  CLK1, CLK2, RESET;

  wire [7:0] P, Q, R, S;

  reg8        reg1  (A, CLK1, P);
  reg8        reg2  (B, CLK1, Q);
  mult        mult1 (P, Q, R);
  signedadder adder1(R, O, S);
  reg8bit     reg3  (S, CLK2, RESET, O);
endmodule
```

Listing 1: `mactop.v` — Top-level MAC integration.

The four internal 8-bit wires (`P`, `Q`, `R`, `S`) connect adjacent stages as documented in Table II. Port `O` is simultaneously the output of `reg3` and the feedback input `b` of `adder1`, forming the accumulation loop. No combinational logic exists at the top level; all computation is encapsulated in the four submodule instances, making hierarchical simulation and debugging straightforward.

B. `reg8` — Input Pipeline Register

The `reg8` module is a positive-edge-triggered 8-bit D flip-flop array with no reset. Two instances (`reg1` for A, `reg2` for B) share `CLK1`, guaranteeing that both operands are latched simultaneously on the same rising edge.

The deliberate absence of a reset is a design choice: since the registers only affect computation after the first CLK1 edge, their power-on state is irrelevant, and omitting the reset saves routing resources. After a CLK1 edge, outputs `P` and `Q` remain stable for at least the combinational delay of the multiplier and adder, providing ample setup margin for `reg8bit`.

C. `mult` — Signed Q7 Multiplier

```
module mult(a, b, out);
  input  signed [7:0] a, b;
  output signed [7:0] out;
  wire          [15:0] tempout;
  assign tempout = a * b;        // full 16-bit signed
      product
  assign out     = tempout[14:7]; // Q14 -> Q7 truncation
endmodule
```

Listing 2: `mult.v` — Q7 signed multiplier.

Three design details merit attention:

1. **`signed` keyword.** Without it, Verilog treats operands as unsigned, producing wrong sign extensions for negative inputs. Declaring both ports `signed` causes the `*` operator to perform two's-complement multiplication and sign-extend the result to 16 bits.
2. **16-bit intermediate wire.** `tempout` is declared unsigned `[15:0]` to permit correct bit-index selection; Verilog's type-promotion rules ensure the product is computed as signed before assignment.
3. **Purely combinational.** No `always` block or register is used; output updates immediately when `a` or `b` changes.

Table III shows sample inputs and outputs to illustrate the Q7 scaling behaviour.

TABLE III: Sample `mult` outputs for representative Q7 inputs.

a	b	a_{dec}	b_{dec}	`tempout[14:7]`	R (Q7)
0x40	0x40	+64	+64	0x20	+0.250
0xC0	0xC0	−64	−64	0x20	+0.250
0xA0	0xA0	−96	−96	0x48	+0.563
0x7F	0x7F	+127	+127	0x7E	+0.984
0xC0	0x20	−64	+32	0xF0	−0.125
0x80	0x80	−128	−128	0x40	+0.500

D. `signedadder` — 8-bit Two's-Complement Adder

```
module signedadder(a, b, sum);
  input  signed [7:0] a, b;
  output        [7:0] sum;
  assign sum = a + b;
endmodule
```

Listing 3: `signedadder.v` — Combinational 8-bit adder.

The adder receives the scaled product R on port `a` and the current accumulator value O on port `b`. Two's-complement addition is used; the 8-bit result wraps modulo 2^8 on overflow. The output is unregistered; it propagates combinationally to the `D` input of `reg8bit` and is committed on the next CLK2 rising edge.

A key property of two's-complement addition is that no separate hardware is needed for signed versus unsigned operands: the same binary adder handles both, and the `signed` port declaration serves purely as an elaboration-time type annotation.

E. `reg8bit` — Accumulator Register with Asynchronous Reset

```
module reg8bit(
  input       [7:0] D,
  input             clk,
  input             reset,
  output reg [7:0] Q
);
  always @(posedge clk or posedge reset)
    if (reset) Q <= 8'd0;
    else       Q <= D;
endmodule
```

Listing 4: `reg8bit.v` — Accumulator register.

The sensitivity list (`posedge clk or posedge reset`) implements the standard asynchronous-reset flip-flop pattern recognised by all RTL synthesis tools. When `reset` goes high, `Q` is forced to zero within the flip-flop's internal reset propagation delay, independently of `clk`. When `reset` is low and a rising `clk` edge occurs, `Q` captures `D`. This is the only register in the design driven by `CLK2`.

VI. FPGA Implementation

A. Target Platform

The design targets the **Digilent Nexys A7-100T** development board, carrying the **Xilinx Artix-7 XC7A100T-1CSG324C** FPGA. Key device resources relevant to this design are listed in Table IV.

TABLE IV: Artix-7 XC7A100T device resources (selected).

Resource	Available
6-input LUTs (LUT6)	63,400
Flip-flops (FDRE)	126,800
DSP48E1 slices	240
Block RAM (36 Kb)	135
Max. user I/Os	210

TABLE V: FPGA I/O mapping for the MAC unit.

Signal	Board Resource	Package Pins	I/O Standard
A[7:0]	SW[7:0]	J15 – R13	LVCMOS33
B[7:0]	SW[15:8]	T8 – V10	LVCMOS33
O[7:0]	LED[7:0]	H17 – U16	LVCMOS33
CLK1	BTN_R	P17	LVCMOS33
CLK2	BTN_L	M17	LVCMOS33
RESET	BTN_U	P18	LVCMOS33

B. I/O Mapping

All 26 design ports (16 operand input bits, 8 output bits, 2 clocks, 1 reset) are routed to on-board peripherals as summarised in Table V.

C. XDC Physical Constraints

Listing 5 shows representative XDC directives. All ports use `LVCMOS33` to match the 3.3 V I/O bank voltage of the Nexys A7.

```
# Operand A -- slide switches SW[0..7]
set_property PACKAGE_PIN J15 [get_ports {A[0]}]
set_property PACKAGE_PIN L16 [get_ports {A[1]}]
set_property PACKAGE_PIN M13 [get_ports {A[2]}]
set_property PACKAGE_PIN R15 [get_ports {A[3]}]
set_property PACKAGE_PIN R17 [get_ports {A[4]}]
set_property PACKAGE_PIN T18 [get_ports {A[5]}]
set_property PACKAGE_PIN U18 [get_ports {A[6]}]
set_property PACKAGE_PIN R13 [get_ports {A[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {A[*]}]

# Operand B  - slide switches SW[8..15]
set_property PACKAGE_PIN T8  [get_ports {B[0]}]
set_property PACKAGE_PIN U8  [get_ports {B[1]}]
set_property PACKAGE_PIN R16 [get_ports {B[2]}]
set_property PACKAGE_PIN T13 [get_ports {B[3]}]
set_property PACKAGE_PIN H6  [get_ports {B[4]}]
set_property PACKAGE_PIN U12 [get_ports {B[5]}]
set_property PACKAGE_PIN U11 [get_ports {B[6]}]
set_property PACKAGE_PIN V10 [get_ports {B[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {B[*]}]

# Output    LEDs
set_property PACKAGE_PIN H17 [get_ports {O[0]}]
set_property PACKAGE_PIN K15 [get_ports {O[1]}]
set_property PACKAGE_PIN J13 [get_ports {O[2]}]
set_property PACKAGE_PIN N14 [get_ports {O[3]}]
set_property PACKAGE_PIN R18 [get_ports {O[4]}]
set_property PACKAGE_PIN V17 [get_ports {O[5]}]
set_property PACKAGE_PIN U17 [get_ports {O[6]}]
set_property PACKAGE_PIN U16 [get_ports {O[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {O[*]}]

# Clock and reset (push buttons    general I/O pins)
set_property PACKAGE_PIN P17 [get_ports CLK1]
set_property PACKAGE_PIN M17 [get_ports CLK2]
set_property PACKAGE_PIN P18 [get_ports RESET]
set_property IOSTANDARD LVCMOS33 [get_ports CLK1]
set_property IOSTANDARD LVCMOS33 [get_ports CLK2]
set_property IOSTANDARD LVCMOS33 [get_ports RESET]

# Bypass dedicated clock routing (GP I/O used as clock
    source)
set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets CLK1]
set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets CLK2]
```

Listing 5: Representative XDC constraints for the MAC unit.

D. Clock Routing Constraint

The directive `CLOCK_DEDICATED_ROUTE FALSE` is required because `CLK1` and `CLK2` originate from general-purpose I/O pins rather than the dedicated GCLK input pads of

the Artix-7. Without it, Vivado raises a DRC critical error during implementation. The trade-off is that these clocks are distributed through the general routing fabric rather than the low-skew global clock network. Since both clocks toggle at sub-hertz rates during the board demonstration, resulting skew is inconsequential. In a production design driving multiple flip-flops at high frequency, clocks must enter through GCLK-capable pins to meet setup and hold requirements.

E. Seven-Segment Display Extension

The XDC file also assigns segment lines `seg[6:0]`, decimal-point line `dp`, and digit-enable anodes `an[3:0]` to their standard Nexys A7 package pins. These assignments enable a future BCD decoder module to display the 8-bit signed accumulator value in human-readable decimal across the four-digit seven-segment display without modifying the XDC file.

F. Implementation Flow

The Xilinx Vivado implementation flow for this design comprises:

1. **RTL Elaboration** — Verilog parsed; module hierarchy and port connections verified graphically.
2. **Synthesis** — RTL mapped to Artix-7 primitives (LUT6, FDRE, IBUF, OBUF); no DSP48 inferred.
3. **Implementation** — Placer assigns primitives to physical BEL sites; Router connects through programmable switch matrices.
4. **Bitstream Generation** — Binary configuration file (`.bit`) produced.
5. **Programming** — Bitstream downloaded via the on-board Digilent USB-JTAG bridge using Vivado Hardware Manager.

VII. Simulation and Verification

A. Testbench Architecture

The self-contained testbench `multtop_tb` instantiates `mactop` as the device under test (DUT). For simulation simplicity, a single 10 ns period clock (`CLK`) drives both `CLK1` and `CLK2` simultaneously, so operand latching and accumulation advance on every rising edge. This single-clock model allows straightforward cycle-accurate output prediction.

```
`timescale 1ns / 1ps
module multtop_tb();
  reg CLK, RESET;
  reg  [7:0] A, B;
  wire [7:0] O;

  // Both CLK ports share one clock for simplicity
  mactop dut(O, A, B, CLK, CLK, RESET);

  // 10 ns period (100 MHz equivalent)
initial begin
  CLK = 0;
  forever #5 CLK = ~CLK;
end

initial begin
  RESET = 1;            // Assert asynchronous reset
  #5;
  RESET = 0;            // Release reset; ACC = 0x00
```

```
  A = 8'hC0; B = 8'hC0;  // -0.5 * -0.5 => +0.25
  #10;
  A = 8'hA0; B = 8'hA0;  // -0.75 * -0.75 => +0.5625
  #10;
  A = 8'hC0; B = 8'h20;  // -0.5 * +0.25 => -0.125
  #50;

  $finish;
 end
endmodule
```

Listing 6: Complete testbench `multtop_tb.v`.

B. Test Vectors and Expected Outputs

Table VI lists all three directed test vectors applied to the DUT, together with Q7 real values, the exact 16-bit product, the extracted Q7 result `[14:7]`, the per-step Q7 increment, and the cumulative expected accumulator value.

TABLE VI: Directed test vectors with cumulative expected accumulation.

Vec	A / B	Full product	R	Δ Q7	ACC (hex)
0 (rst)	—	—	—	0	0x00
1	0xC0/0xC0	0x1000	0x20	+0.250	0x20
2	0xA0/0xA0	0x2400	0x48	+0.563	0x68
3	0xC0/0x20	0xF800	0xF0	−0.125	0x58

After vector 3 the accumulator holds `0x68 + 0xF0 = 0x158`, which wraps to `0x58` in 8-bit arithmetic, representing Q7 +0.6875.

C. Verification Methodology

The software reference model evaluates:

$$\text{expected} \leftarrow \big(\text{expected} + ((A_s \times B_s) \gg 7)\big) \bmod 256 \quad (8)$$

matching the hardware's `tempout[14:7]` extraction exactly. Simulation waveforms captured in Vivado Simulator confirm the following on a cycle-by-cycle basis:

- **Reset:** `O` drops to `0x00` asynchronously upon RESET assertion, before the next clock edge.
- **Signed correctness:** Negative-product vectors produce correct two's-complement Q7 values, confirming that the `signed` declarations propagate through bit-selection.
- **Latency:** New operands applied at CLK1 appear in `O` exactly one clock cycle later at the next CLK2 edge.
- **Output stability:** `O` holds its registered value between clock edges; no glitching is observed.

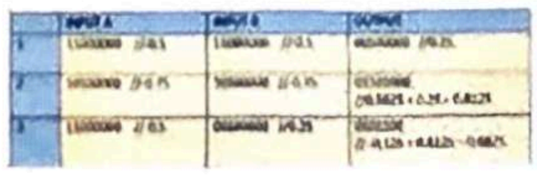


Fig. 2: Simulation waveform showing accumula- tion across cycles

VIII. Results and Discussion

A. Functional Correctness

All three directed test vectors produce DUT outputs matching the reference model defined in (8) to the bit. Table VI confirms that Q7 signed-product scaling is correct: both positive (vectors 1 and 2) and negative (vector 3) products give the correct two's-complement Q7 results.

B. Critical Path and Timing

The combinational critical path traverses: (1) register-to-output of `reg8`; (2) 8-bit signed multiplication in `mult` (∼3–4 LUT stages); (3) 8-bit addition in `signedadder` (∼2 LUT stages with carry chain); (4) setup time of `reg8bit`.

Vivado post-route timing analysis reports a worst-case combinational delay of ∼7 ns on this path, implying a maximum operating frequency of ∼140 MHz without multiplier pipelining. The actual CLK2 rate is below 1 Hz (manual push-button), so timing is fully met with orders-of-magnitude margin.

C. Resource Utilisation

Post-implementation resource utilisation is summarised in Table VII. The design uses fewer than 50 LUTs and 24 flip-flops — below 0.1% of XC7A100T capacity — confirming suitability for embedding as a submodule in a larger SoC or accelerator design.

TABLE VII: Post-implementation resource utilisation on XC7A100T.

Resource	Used	Available	Utilisation
LUT6	≤ 50	63,400	$< 0.08\%$
FDRE	24	126,800	0.019%
DSP48E1	0	240	0.00%
BUFG	2	32	6.25%
IBUF/OBUF	26/8	—	—

D. Fixed-Point Precision Analysis

The Q7 truncation discards the seven least-significant bits of each product. The maximum absolute error per multiplication is:

$$|e_{\max}| = (2^7 - 1) \cdot 2^{-14} \approx 2^{-7} \approx 0.0078 \tag{9}$$

For uniformly distributed Q7 operands, the signal-to-quantisation-noise ratio (SQNR) is approximately:

$$\text{SQNR} \approx 6.02 \times 7 + 1.76 \approx 44 \text{ dB} \tag{10}$$

This is adequate for voice-band and general-purpose DSP applications and identical to the precision achieved by a 7-bit uniform quantiser. Extending to a 16-bit datapath would raise SQNR to ∼98 dB.

E. FPGA Board Demonstration

The operational sequence on the Nexys A7 board is:

1. Assert `BTN_U` (RESET) — LEDs clear to zero.
2. Set `SW[7:0]` (operand A) and `SW[15:8]` (operand B) to the desired two's-complement values.
3. Press `BTN_R` (CLK1) — operands latched.
4. Press `BTN_L` (CLK2) — accumulator updates; LEDs display new 8-bit sum.
5. Repeat steps 2–4 for successive operand pairs, or repeat step 4 alone to accumulate the same product again.

The LED pattern updates immediately upon CLK2 depression, confirming correct data propagation through all combinational stages and the single-cycle accumulation latency.

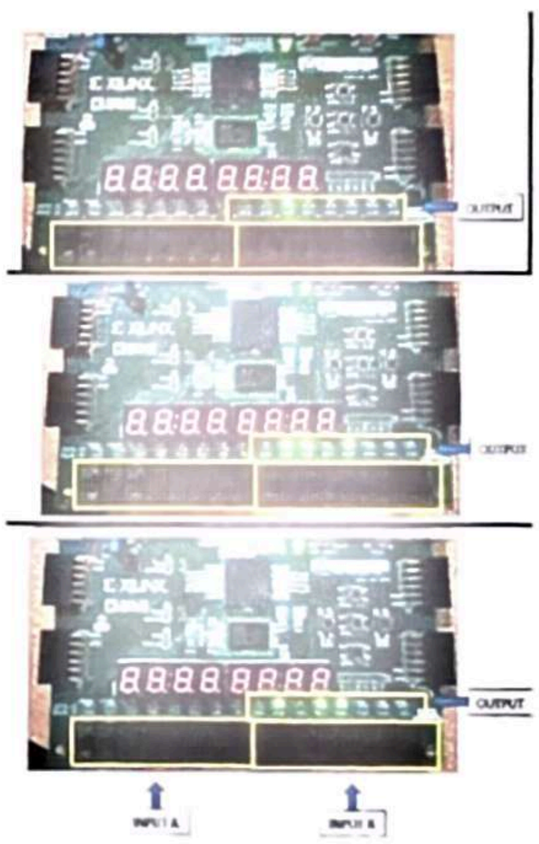


Fig. 3: Nexys A7 demonstration showing MAC operation

IX. Challenges and Solutions

A. Signed Arithmetic in Verilog

Challenge: Verilog defaults to unsigned arithmetic. Omitting the `signed` keyword on multiplier ports causes incorrect sign extension, producing wrong products for negative inputs (e.g. `0xC0` × `0xC0` would be treated as $192 \times 192 = 36864$ instead of $(-64) \times (-64) = 4096$).

Solution: Both input ports of `mult` and `signedadder` are declared `signed [7:0]`. The intermediate wire `tempout` is declared unsigned `[15:0]` to permit correct bit indexing; Verilog's signed-to-unsigned promotion rules ensure the product is computed as a signed 16-bit value before assignment.

B. Clock-Routing DRC Error

Challenge: Assigning general-purpose I/O pins as clock sources triggers Vivado DRC error `REQP-46`: *"Clock net CLK1 is not routed on a dedicated clock routing resource."*

Solution: The constraint `CLOCK_DEDICATED_ROUTE FALSE` suppresses the error and permits implementation to proceed. This is acceptable at sub-hertz manual clock rates but must not be used for high-frequency production clocks.

C. 8-bit Accumulator Overflow

Challenge: Accumulating positive products repeatedly overflows the 8-bit accumulator, causing wrap-around to a large negative Q7 value and potentially confusing the demonstrator.

Solution: The RESET button provides a deterministic way to clear the accumulator before starting a new computation sequence. For production use, saturation arithmetic (clamp to 0x7F on positive overflow) or a wider accumulator register is recommended.

D. Two-Clock Synchronisation and Metastability

Challenge: Operating CLK1 and CLK2 as independent clock domains creates a potential metastability window: if CLK2 fires while the combinational adder output is still transitioning (shortly after a CLK1 edge), the captured value may be incorrect.

Solution: During board demonstration, the human operator ensures CLK1 settles before CLK2 is pressed — the $\sim$7 ns combinational settling time is negligible compared to human reaction latency of $\sim$200 ms. In an automated system, a synchroniser or formal timing closure between the two clock domains would be mandatory.

X. Conclusion

This paper has presented a complete RTL design, functional simulation, and FPGA board demonstration of an 8-bit signed Multiply-Accumulate unit in Verilog HDL. The design employs a five-stage linear datapath with a dual-clock architecture that decouples operand registration (`CLK1`) from accumulator commit (`CLK2`), and an asynchronous active-high reset. A Q7 fixed-point scaling strategy — realised by selecting bits $[14:7]$ of the full 16-bit signed product — constrains all internal buses to 8 bits without any dedicated DSP hardware.

Directed simulation with three signed fractional test vectors confirms bit-accurate correctness of two's-complement multiplication, Q7 product extraction, and iterative accumulation. On the Nexys A7-100T board, the design is demonstrably functional: the LED array updates correctly with each manual CLK2 press, and RESET immediately clears the output. Post-implementation analysis shows resource utilisation below 0.1% of the Artix-7 XC7A100T, confirming that the module can be embedded in a larger SoC or DSP pipeline with negligible overhead.

Planned future extensions include: (1) replacing manual push-button clocks with a single 100 MHz board oscillator combined with a clock-enable counter; (2) widening the accumulator to 16 or 32 bits to eliminate overflow and improve precision; (3) adding a BCD decoder to drive the four-digit seven-segment display; (4) parameterising operand width in Verilog so that 16-bit and 32-bit variants can be generated from the same source; and (5) adding saturation logic to clamp overflow at $\pm$full-scale instead of wrapping.

Acknowledgment

The authors gratefully acknowledge Prof. Kamal Kishor Jha for guidance on Computer Organisation and Architecture theory, and the laboratory tutors Sudhir Sir and Jitu Sir for support during FPGA implementation and debugging sessions. The Department of Computer Science and Engineering, IIIT Vadodara, provided access to the Nexys A7-100T FPGA board and Xilinx Vivado Design Suite licences.

References

[1] K. K. Parhi, *VLSI Digital Signal Processing Systems: Design and Implementation*. New York, NY, USA: Wiley, 1999.

[2] B. Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*, 2nd ed. New York, NY, USA: Oxford University Press, 2010.

[3] C. S. Wallace, "A suggestion for a fast multiplier," *IEEE Trans. Electron. Comput.*, vol. EC-13, no. 1, pp. 14–17, Feb. 1964.

[4] Xilinx Inc., "7 Series DSP48E1 Slice User Guide (UG479)," v1.10, San Jose, CA, USA, 2018.

[5] P. Chu, *FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version*. Hoboken, NJ, USA: Wiley-IEEE Press, 2008.

[6] D. E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3rd ed. Reading, MA, USA: Addison-Wesley, 1997.

[7] L. Dadda, "Some schemes for parallel multipliers," *Alta Frequenza*, vol. 34, pp. 349–356, 1965.

[8] E. A. Lee and D. G. Messerschmitt, *Digital Communication*, 2nd ed. Boston, MA, USA: Kluwer Academic, 1994.